

第 29 讲

Autograd: `loss.backward()` 到底怎样自动得到梯度

从手工 Backpropagation 过渡到 PyTorch 自动微分

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】第 22–26 讲已经把 Chain Rule、Computational Graph、单神经元反向传播、两层 MLP Backpropagation 和完整 Training Loop 手工走通。现在开始回答：PyTorch 的 Autograd 到底自动替我们做了哪几步？目标不是“会写 `loss.backward()`”，而是看到它时能在脑中还原出：记录计算图 → 逆序遍历 → 局部导数 → Chain Rule → 分支梯度累加 → 写入 `.grad`。

学习目标

学完本讲，你应该能够：

1. 解释 `requires_grad=True` 到底开启了什么；
2. 区分 leaf Tensor 与 non-leaf Tensor；
3. 解释 `grad_fn` 表示什么；
4. 说明 `loss.backward()` 为什么通常从标量 Loss 开始；
5. 知道梯度最终为什么出现在参数的 `.grad` 中；
6. 解释为什么 `.grad` 默认累加，以及为什么训练循环必须清梯度；
7. 用 Autograd 复现第 24 讲单神经元手工梯度；
8. 用矩阵公式复核 `nn.Linear` + MSE 的梯度；
9. 理解 `torch.no_grad()` 与 `detach()` 的最小区别；
10. 理解 backward 只负责“算梯度”，并不会自动更新参数。

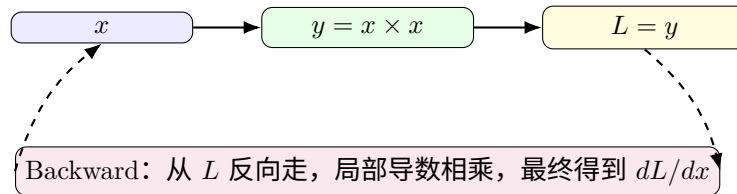
1 【原理】Autograd 没有发明新的数学

前面我们已经知道 Backpropagation 的核心是：

Computational Graph + Chain Rule + Gradient Accumulation

PyTorch Autograd 做的是把这些步骤自动化：

1. Forward 时记录哪些 Tensor 由哪些操作产生；
2. 从最终目标开始逆序遍历计算图；
3. 调用每个操作对应的局部 backward 规则；
4. 沿路径应用 Chain Rule；
5. 多条路径汇合时累加梯度；
6. 把需要保留的梯度写入对应 Tensor 的 `.grad`。



Autograd 的价值不是“数学变了”，而是我们不需要再手工维护巨大计算图中每一个中间变量的反向公式。

2 【第一步】requires_grad=True 到底做了什么？

假设：

```
x = torch.tensor(3.0, requires_grad=True)
y = x * x
z = y + 2.0
```

这里 `requires_grad=True` 的含义是：

如果后续运算依赖于 x ，请保留足够的信息，以便将来能计算某个最终目标对 x 的梯度。

它不代表梯度已经算出来了。在执行 `backward()` 前：

```
x.grad=None
```

常见误区：`requires_grad=True` $\neq$ “自动开始训练”。它只是让相关计算进入 Autograd 跟踪。真正反向计算要等到调用 `backward()` 或 `autograd.grad()`。

3 【Leaf / Non-leaf】为什么参数有 `.grad`，中间结果默认没有？

在最小例子中：

- x 是用户直接创建的、需要梯度的 Tensor，因此通常是 leaf Tensor；
- $y = x^2$ 是某个操作产生的中间结果，因此是 non-leaf Tensor；
- $z = y + 2$ 同样是 non-leaf Tensor。

实验中：

```
x.is_leaf -> True
x.grad_fn -> None

y.is_leaf -> False
y.grad_fn -> MulBackward0

z.grad_fn -> AddBackward0
```

grad_fn 可以理解成：

“这个中间 Tensor 是由哪一种可微操作产生的，因此 backward 时要调用哪类局部反向规则？”

默认情况下，PyTorch 主要把梯度保存在需要梯度的 leaf Tensor 上。模型的 Parameter 通常就是这种叶子节点。

以后看到 `parameter.grad`，你应该理解为：最终 Loss 对这个可训练参数的偏导数，经过整张计算图的 Chain Rule 后被累积到了这里。

4 【最小数字例子】backward() 如何得到 6？

令：

$$L = x^2, \quad x = 3$$

手工已经知道：

$$\frac{dL}{dx} = 2x = 6$$

PyTorch：

```
x = torch.tensor(3.0, requires_grad=True)
loss = x ** 2
loss.backward()
print(x.grad)  # tensor(6.)
```

从计算图角度：

$$x \xrightarrow{\times x} L$$

Backward 时，乘法节点的局部规则给出：

$$\frac{dL}{dx} = 2x$$

代入当前 Forward 保存的 $x = 3$ ，得到 6。

这就是第 13 讲“导数是当前点的局部斜率”和第 23 讲“节点局部 backward 规则”进入 PyTorch 后的同一个东西。

5 【关键实验】第 24 讲单神经元，Autograd 能否复现手工梯度？

我们重新使用：

$$x = [2, -1, 3], \quad w = [0.5, -2, 1], \quad b = 0.2, \quad y = 5$$

Forward：

$$z = x \cdot w + b = 6.2$$

$$a = \text{ReLU}(z) = 6.2$$

$$L = (a - y)^2 = 1.44$$

第 24 讲手工得到：

$$\frac{\partial L}{\partial w} = [4.8, -2.4, 7.2]$$

$$\frac{\partial L}{\partial b} = 2.4$$

$$\frac{\partial L}{\partial x} = [1.2, -4.8, 2.4]$$

本讲用：

```
loss.backward()
print(w.grad)
print(b.grad)
print(x.grad)
```

得到相同结果，仅有 float32 舍入误差，约 10^{-6} 量级。

【验证思想】以后自己实现新 Layer、新 Loss 或复杂机制时，常见的最低层验证方法之一就是：
小规模手工/解析梯度 $\leftrightarrow$ Autograd $\leftrightarrow$ 数值差分。三者对齐，才能更有把握地继续扩大实验。

6 【为什么标量 Loss 最方便】非标量输出直接 backward 为什么会报错？

假设：

$$y = [x_1^2, x_2^2, x_3^2]$$

此时输出不是一个标量，而是一个向量。问题变成：

“你究竟要对这三个输出怎样加权，才能得到最终要反传的那个目标？”

所以：

```
y.backward()
```

会报错，因为 PyTorch 无法凭空猜测上游梯度。

最常见做法是先聚合：

```
loss = y.sum()
loss.backward()
```

如果明确给出上游梯度，也可以使用：

```
torch.autograd.grad(
    y, x,
    grad_outputs=upstream
)
```

这和第 23 讲的“每个节点收到一个上游梯度，再乘本地导数”完全一致。非标量输出只是需要你明确告诉系统：来自后续的上游梯度是什么。

7 【最容易踩坑】为什么梯度默认会累加？

设：

$$p = 2, \quad L = p^2$$

第一次：

```
loss = p ** 2
loss.backward()
# p.grad = 4
```

如果不清空梯度，再重新 Forward + backward：

```
loss = p ** 2
loss.backward()
# p.grad = 8
```

为什么不是 4？

因为 PyTorch 默认执行的是类似：

$$grad \leftarrow grad + new_grad$$

而不是覆盖。

这是有意设计的，因为同一个参数可能被多条路径、多个 micro-batch 等共同使用，梯度累加是有意义的。

训练循环里通常要：

```
optimizer.zero_grad()
# 或 model.zero_grad()
```

然后再做下一次：

```
loss.backward()
```

如果忘记清梯度，训练可能不会立即报错，但参数收到的梯度会混入前一轮贡献。这类 Bug 比“程序崩溃”更危险，因为它会悄悄改变实验。

8 【计算图是动态的】为什么第二次 backward 常常要重新 Forward?

PyTorch 常被称为动态计算图框架：你每次执行 Python Forward 代码，就会按照当次实际执行的操作建立一张新图。

通常一次：

```
loss.backward()
```

之后，为了节省内存，中间反向所需的一些缓存会被释放。因此下一训练 Step 通常是：

```
pred = model(x)      # 新 Forward, 新计算图
loss = criterion(pred, target)
loss.backward()       # 对这张新图 backward
```

如果确实需要在同一张图上重复 backward，可以了解：

```
loss.backward(retain_graph=True)
```

但这不是普通训练循环的默认做法，而且会增加内存占用。

9 【矩阵版】nn.Linear + MSE 的 Autograd 到底算了什么?

设 PyTorch Linear：

$$Y = XW^T + b$$

其中：

$$X : (B, D_{in}), \quad W : (D_{out}, D_{in}), \quad Y : (B, D_{out})$$

如果 MSE 对所有输出元素求 mean，先有：

$$dY = \frac{2(Y - T)}{N}$$

其中 N 是输出元素总数。

然后：

$$dW = dY^T X$$

$$db = \sum_{batch} dY$$

$$dX = dY W$$

实验中 Autograd 与这三个手工矩阵公式最大误差全部为：

0

因此 nn.Linear 的 Autograd 并不是“黑盒神奇求导”。它只是在自动执行你第 25 讲已经学过的矩阵 Backpropagation。

10 【Shape 规则】Gradient 为什么和原变量同 Shape?

本实验：

$$W.shape = (2, 3)$$

所以：

$$W.grad.shape = (2, 3)$$

$$b.shape = (2,)$$

所以：

$$b.grad.shape = (2,)$$

$$X.shape = (2, 3)$$

所以：

$$X.grad.shape = (2, 3)$$

原因很直接：梯度必须告诉我们原变量**每一个元素**轻微变化时，Loss 如何变化。

11 【no_grad 与 detach】什么时候不希望记录梯度?

1. torch.no_grad()

它表示：

“在这一段代码执行期间，不要建立 Autograd 跟踪。”

常见于：

- 参数手工更新；
- 纯推理；
- 某些不需要梯度的辅助计算。

```
with torch.no_grad():  
    parameter -= lr * parameter.grad
```

2. detach()

它针对一个 Tensor，返回一个与原计算图断开的 Tensor 视图：

```
y_detached = y.detach()
```

新的 Tensor：

- 不再沿原图反传；
- 常用于记录、可视化、缓存或明确阻断梯度流。

初学阶段先把二者理解成“停止继续追踪梯度”的两种不同工具即可。不要为了让报错消失而随意 `detach()`，因为它可能无意中切断本应存在的训练信号。

12 【非常重要】`backward()` 不等于参数更新

执行：

```
loss.backward()
```

做的事情是：

计算并累积 Gradient

它不会自动改变 Weight/Bias。

要更新参数，还需要：

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

本讲实验仍然手工写：

```
with torch.no_grad():
    weight -= lr * weight.grad
    bias   -= lr * bias.grad
```

后面的 `Optimizer` 课程会把这一步封装成：

```
optimizer.step()
```

13 【完整训练 Step 预览】PyTorch 实际上把前 26 讲压缩成了什么？

```
optimizer.zero_grad()    # 清上一轮梯度
prediction = model(x)     # Forward，动态建立计算图
loss = criterion(prediction,y) # 构造标量目标
loss.backward()          # 自动 Backprop，填充 parameter.grad
optimizer.step()         # 根据 grad 更新参数
```

你现在应该能把每一行还原成底层：

<code>zero_grad</code>	清除旧的梯度累积，避免混入下一 Step
<code>model(x)</code>	执行矩阵运算、激活等 Forward，并记录依赖图
<code>criterion</code>	把 Prediction 与 Target 变成标量 Loss
<code>backward</code>	逆序遍历图，Chain Rule，累积各参数 Gradient
<code>step</code>	使用 Optimizer 规则真正修改参数

【科研实验】以后如果 Loss 异常、参数不更新、Gradient 为 None/0/NaN，排错时不要只盯着 optimizer。应该沿这个闭环逐段问：参数是否 requires_grad？Forward 是否真的依赖该参数？Loss 是否连接到这条路径？梯度是否被 detach/no_grad 切断？是否忘了 backward？是否清梯度时机错误？

14 【LLM / Transformer 连接】大模型训练为什么同样依赖 Autograd？

LLM Forward 可以抽象为：

Token IDs → Embedding → Transformer Blocks → Logits → Cross Entropy

Cross Entropy 得到标量或聚合后的 Loss 后，Autograd 会把梯度反向传播经过：

Output Projection → MLP → Attention → LayerNorm → ... → Embedding/Parameters

当然，大模型工程中还会加入分布式训练、混合精度、梯度累计、checkpointing 等机制，但“从 Loss 沿计算图反向求参数梯度”这个数学骨架没有改变。

15 【常见误区】

1. **误区：**requires_grad=True 就表示已经有梯度。错。没有 backward 前，.grad 通常仍为 None。
2. **误区：**backward 会自动更新参数。错。它只算梯度；参数更新是 optimizer/manual update 的职责。
3. **误区：**每次 backward 都会覆盖旧梯度。错。默认累加。
4. **误区：**所有中间 Tensor 都会自动保存.grad。错。non-leaf 默认通常不保留.grad；如确有需要可了解 retain_grad()。
5. **误区：**detach 是“让代码运行”的万能工具。错。detach 会改变梯度流，可能破坏训练逻辑。
6. **误区：**Autograd 算出的梯度一定代表实现正确。错。它只对你实际写出的计算图正确求导；如果 Forward 逻辑写错，梯度也会忠实地对错误逻辑求导。

16 【科研阅读与 Ablation】Autograd 本身通常不是论文创新点，但它决定实验是否可信

做 Agent / LLM Systems 研究时，你可能不常改底层反向公式，但仍应具备以下最低判断力：

- 哪些参数是 frozen，哪些参数 requires_grad；
- LoRA / Adapter 到底有哪些参数接收梯度；
- 某个 memory / retrieval / routing 模块是否可微；
- stop-gradient / detach 是否切断某条训练信号；

- gradient accumulation 是否改变有效 Batch Size;
- gradient checkpointing 是计算/显存策略, 不是“改变数学目标”;
- 训练失败是 Forward、Loss、Gradient、Optimizer 还是数据环节的问题。

本讲最低验收标准

如果你能不看答案解析下面 8 句话, 本讲就达标:

1. requires_grad=True 是“追踪依赖”, 不是“梯度已经存在”;
2. leaf Tensor 常用 .grad 保存最终累积梯度;
3. grad_fn 记录 non-leaf Tensor 来自什么可微操作;
4. loss.backward() 本质是自动执行计算图上的 Backpropagation;
5. 标量 Loss 最方便作为反向传播起点;
6. .grad 默认累加, 因此训练 Step 之间需要清梯度;
7. backward() 只算 Gradient, 不更新参数;
8. Autograd 的底层数学仍然是 Chain Rule。

与下一讲的连接

现在我们已经知道 PyTorch 怎样自动求 Gradient。下一讲进入:

nn.Module	Layer Parameter Forward
-----------	-------------------------

届时会自己写一个 PyTorch MLP 类, 并学习 parameters()、state_dict()、train()/eval() 的最小结构。